

INDEXING PIPELINE

Git 기반 마크다운 RAG 인덱싱 파이프라인

Bitbucket 레포의 마크다운 문서를 주기적으로 수집해
에이전트 검색이 활용할 **Milvus 벡터 인덱스**로 변환·유지합니다.

문제 의식

조직 내부에 분산된 마크다운 지식이 검색 가능한 자산이 되지 못하고 있습니다.

- 1 · 분산

개인 로컬에만 흩어진 문서가 조직 차원에서 검색되지 않음
- 2 · 변동

문서는 끊임없이 바뀌지만 인덱스가 그 속도를 따라잡지 못함
- 3 · 마찰

작성자가 청킹·메타·임베딩까지 신경 써야 하면 아무도 쓰지 않음

작성자는 올리기만, 나머지는 파이프라인이

01 공용 레포 · 개인 디렉토리

개인 지식을 한 곳에 모아 조직이 함께 검색할 수 있게 합니다.

02 자동 동기화

문서가 바뀌어도 색인이 자동으로 따라옵니다.

03 낮은 인지 부하

작성자는 .md 만 올리면 끝 — 청킹·캡션·벡터화는 전부 자동.

04 이동에 강건

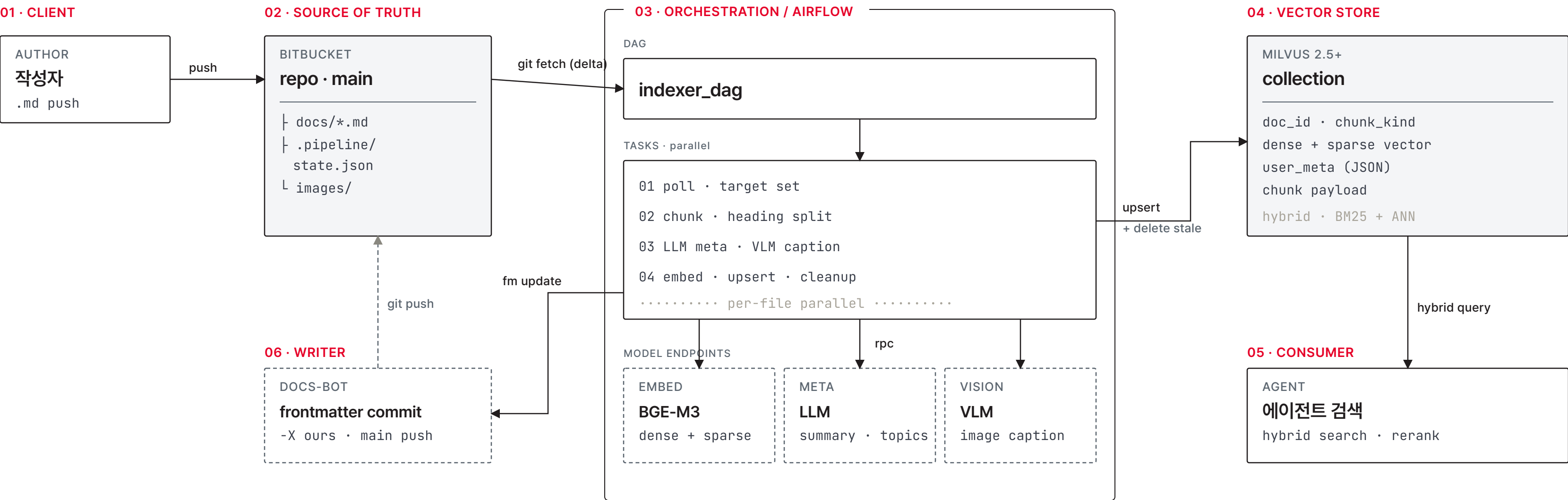
파일을 옮기거나 이름을 바꿔도 doc_id 가 유지되어 재임베딩이 불필요합니다.

05 본문은 사용자 소유

파이프라인은 프론트매터만 건드리므로 git blame 이 흐려지지 않습니다.

전체 아키텍처

컴포넌트 · 데이터 흐름 · 쓰기 루프



파이프라인 한 사이클

Airflow DAG 한 번의 실행이 수행하는 다섯 단계.

01 폴링 main 브랜치 fetch, 마지막 색인 이후 diff 확인	02 대상 수집 변경된 .md · 이미지가 바뀐 .md · 실패 재시도	03 가공 병렬 청킹 · VLM 캡션 · LLM 메 타 추출	04 임베딩 · 적재 BGE-M3 dense+sparse upsert, Stale 청크 제거	05 봇 커밋 프론트매터와 state.json을 한 커밋으로 main에 push
--	--	--	--	---

RESILIENCE 실패한 문서는 `indexing_status: failed` 로 기록되어 다음 실행에서 자동 재시도됩니다. 한 문서의 실패가 전체 실행을 막지 않습니다.

무엇을 이번 실행에서 처리할 것인가

SET A

변경된 .md

마지막 색인 커밋 이후 수정·추가·이동된 문서

SET B

이미지가 바뀐 .md

참조 이미지가 갱신되면 해당 문서도 재처리 대상

SET C

이전에 실패한 .md

재시도 큐 — 이번 실행에서 다시 처리

target = A ∪ B ∪ C

문서 가공 단계

청킹

heading 기반 분할

문서 구조를 살려 의미 단위로 쪼갭니다. 섹션 경계가 검색 정답과 잘 맞도록 정렬.

이미지 · VLM

캡션 생성 → images

본문에 삽입된 이미지는 VLM이 캡션을 만들어 프론트매터 `images` 필드에 기록됩니다.

메타 · LLM

요약 · 분류 · 언어

`summary` , `doc_type` , `topics` , `language` 를 LLM이 자동으로 추출합니다.

PARALLEL 대상 문서마다 독립적으로 처리되어, 문서 하나의 지연이 다른 문서 처리를 막지 않습니다.

하이브리드 임베딩과 Stale 청크 정리

BGE-M3

dense + sparse 두 벡터를
같은 모델에서 한 번에 생성

Milvus 2.5+의 하이브리드 검색이 두 벡터를 결합해 의미 + 키워드 매칭을 함께 활용합니다.

UPSERT + CLEANUP

같은 `doc_id`의 결과를
먼저 upsert, **Stale 청크**는 삭제

이전엔 있었으나 이번 청킹 결과엔 없는 과거 조각. 정리하지 않으면 검색 결과에
유령 청크가 섞입니다.

예시 문서가 축약돼 청크 개수가 **12 → 8**로 줄면, 9~12번 자리에 남아 있던 옛 청크가 Stale이 되어 이번 단계에서 제거됩니다.

봇 커밋과 충돌 정책

한 커밋에 묶어서

갱신된 프런트매터와 `.pipeline/state.json` 을 `docs-bot` 계정으로 **한 커밋**에 묶어 main에 push.

충돌 시 봇 우선

푸시 충돌이 발생하면 프런트매터 예약 필드는 **봇이 방금 만든 값**을 **우선** 적용합니다.

```
rebase -X ours
```

BY DESIGN 봇은 본문을 절대 수정하지 않으므로 본문 충돌은 원천적으로 발생하지 않습니다. 예약 필드를 사용자가 수정한 상태로 머지되어도 다음 실행에서 최신 색인 결과가 덮어씹힙니다.

레포에 생기는 변화

A **프론트매터 자동 추가·갱신**
각 `.md` 상단에 예약 필드가 파이프라인에 의해 생성·유지됩니다.

B **.pipeline/ 디렉토리 생성**
레포 루트에 봇 전용 디렉토리가 생깁니다. 수동 편집 금지.

C **docs-bot 커밋이 히스토리에 등장**
본문은 절대 수정되지 않으므로 `git blame` 은 흐려지지 않습니다.

식별 · 버전

파이프라인이 직접 관리하는 식별자·해시·스키마 버전. 사용자 편집은 다음 색인에서 덮어씁니다.

필드	용도
doc_id	문서의 영속 식별자 (UUID v4). 이동·이름 변경에도 불변
path	현재 레포 내 상대 경로
content_hash	프론트매터를 제외한 본문 해시. 변경 감지 기준
last_indexed_commit	마지막으로 색인된 git 커밋 SHA
last_indexed_at	마지막 색인 시각 (ISO 8601)
pipeline_schema_version	프론트매터 스키마 버전
chunk_strategy_version	청킹 전략 버전
embedding_model	임베딩 모델 식별자
vision_schema_version	이미지 처리 스키마 버전

내용 · Git · 운영 · 이미지

내용 분류 — LLM 자동 생성

title	문서 제목 (첫 H1 또는 LLM 보정)
summary	한 줄 요약 (50~100자)
doc_type	문서 타입 분류
topics	주요 키워드 (3~7개)
language	언어 (ko / en / mixed)

GIT 유래 메타

authors	주요 기여자 (git log 상위 5명)
created_at	최초 커밋 시각
updated_at	최근 본문 변경 커밋 시각

운영 상태

chunk_count	현재 청크 수
has_images	이미지 포함 여부
indexing_status	ok / failed / excluded / pending
web_url	Bitbucket 원문 URL

IMAGES 별도 필드 `images` 는 본문 이미지별 VLM 캡션과 섹션 경로의 배열로 저장됩니다.

사용자 필드와 특수 플래그

사용자 필드

예약 목록에 없는 키는 **사용자 자유 영역**.

팀 메타데이터를 자유롭게 추가할 수 있으며, 파이프라인은 읽기만 하고 그대로 보존합니다.

Milvus에는 `user_meta` JSON 필드에 통째로 저장되어 에이전트 검색의 필터 조건으로 활용 가능합니다.

특수 플래그 · RAG_EXCLUDE

`rag_exclude: true` 면 즉시 색인에서 제외.

→ 기존 청크는 Milvus에서 즉시 삭제

→ 프론트매터는 `indexing_status: excluded` 로 유지

ok

failed

excluded

pending

작성자가 알아야 할 규칙

- 01 색인 대상은 main의 `.md`**
feature 브랜치 상태는 검색에 노출되지 않습니다.
- 02 이미지는 같은 디렉토리 또는 하위에**
외부 URL이나 심볼릭 링크는 처리되지 않습니다.
- 03 이동·이름 변경은 `git mv` 로**
`doc_id` 가 유지되어 재임베딩 없이 메타만 갱신됩니다.
- 04 민감 문서는 `rag_exclude: true`**
프론트매터 한 줄로 색인에서 뺄 수 있습니다.

전제 조건

배포 전 확보해야 할 다섯 가지 구성 요소.

구성 요소	필요 조건
Bitbucket 레포	main push 권한 서비스 계정(docs-bot 등) + SSH 키
Airflow 2.x	사내 클러스터 (DAG 배포 가능)
Milvus 2.5+	하이브리드 검색(dense + sparse)을 지원하는 버전
모델 엔드포인트	BGE-M3 (임베딩) · LLM (요약·분류) · VLM (이미지 캡션)
시크릿 저장소	Vault 또는 K8s Secret — SSH 키, 엔드포인트 토큰

구성 파일과 배포 흐름

PIPELINE-CONFIG.YAML

레포 배포물에 두고 Airflow 워커에서 로드합니다. 주요 키:

repo

milvus

embedding

llm

vision

chunking

schedule

bot

상세 키와 기본값은 설계 문서 §6 참조.

최초 배포 흐름

- 01전제 조건 준비 + 설정 파일 작성
- 02indexer_dag 배포 + SSH 키 마운트
- 03레포 전체 초기 색인

.pipeline/state.json 이 없으므로 전체 .md 를 최초 색인, 봇 커밋으로 프론트매터를 채움
- 04이후엔 커밋 단위 diff만 처리

BACKFILL

모델·청킹 전략·프론트매터 스키마가 바뀔 때는 pipeline-backfill.py 로 Shadow 컬렉션을 만들어 무중단 교체합니다.

범위와 비범위

다루는 것

- Bitbucket 단일 레포 `main` 의 `.md`
- 본문 이미지의 VLM 캡션
- 변경 동기화 — 추가·수정·삭제·이동
- 프론트매터 예약 필드 자동 관리
- 최종 일관성 보장

다루지 않는 것

- 에이전트 검색 구현 — 랭킹·리랭커·UI
- 문서 단위 권한 제어 (레포 단위로 경계)
- `.md` 외 포맷 — Confluence · Notion · PDF · Office
- 외부 URL 이미지 · 심볼릭 링크
- `main` 외 브랜치 · 실시간 반영
- Bitbucket 외의 Git 호스팅

SUMMARY

작성자 **올리기만**, 나머지는 **파이프라인**이.

Git을 단일 소스로 삼아, 마크다운 문서가 검색 가능한 조직 자산이 되기까지의 **전 과정을 자동화**합니다. 본문은 사용자의 것, 메타와 벡터는 파이프라인의 책임.